

## Labo 02 — Réponses commentées

### Question 1 — Noms complets

#### Réponse.

| Écrit                                                    | Interprété par Docker                       |
|----------------------------------------------------------|---------------------------------------------|
| <code>nginx</code>                                       | <code>docker.io/library/nginx:latest</code> |
| <code>bitnami/nginx</code>                               | <code>docker.io/bitnami/nginx:latest</code> |
| <code>registry.masociete.fr:5000/socle/nginx:1.25</code> | tel quel — registry explicite               |

**Pourquoi.** La règle est purement syntaxique : Docker regarde le premier segment, avant le premier `/`. S'il contient un **point** (`.`), un **deux-points** (`:`, donc un port), ou s'il vaut exactement `localhost`, c'est un **registry**. Sinon, c'est un namespace sur Docker Hub. Un nom sans aucun `/` est complété par le namespace `library`.

**Nuance.** Cette règle explique un incident classique : un registry interne nommé `registry` sans domaine (`registry/socle/nginx`) sera interprété comme le namespace Docker Hub `registry`, et le pull échouera en `not found` — ou pire, ramènera une image inconnue publiée par un tiers. Le nom d'un registry interne doit toujours être un FQDN.

#### Exemple.

```
docker pull nginx:alpine
docker image inspect --format '{{.RepoTags}}' nginx:alpine # [nginx:alpine]
docker image inspect --format '{{index .RepoDigests 0}}' nginx:alpine
# nginx@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752
```

### Question 2 — Même tag, contenus différents

**Réponse.** Parce qu'un tag est un **pointeur mouvant**. Entre les deux pulls, quelqu'un a republié `monapp/api:2.3` : le tag pointe maintenant vers un autre contenu, donc un autre digest. A tourne sur l'ancienne image, B sur la nouvelle.

**Pourquoi.** Rien dans le protocole des registries n'interdit de réécrire un tag. Le registry ne stocke qu'une association `tag → digest`, qu'un `push` remplace sans avertissement ni historique.

**Nuance.** Le cas est plus vicieux qu'il n'y paraît : sur A, l'image locale n'a pas bougé, donc aucun symptôme ne vous alerte. Le seul moyen de détecter l'écart est de comparer les **digests** — les `IMAGE ID` (identifiants de configuration locale) diffèrent aussi, mais parlent moins. La bonne pratique est double : tags **immuables** par build, et registry configuré pour interdire l'écrasement d'un tag existant (Harbor et ECR savent le faire).

#### Exemple.

```
# Sur A et sur B :
docker image inspect --format '{{index .RepoDigests 0}}' monapp/api:2.3
# Deux sha256 différents = deux images différentes sous le même tag.
```

### Question 3 — 62 Go de `SIZE` sur un disque de 100 Go

**Réponse.** Non. La colonne `SIZE` de `docker images` donne, pour chaque image, la taille de **toutes ses couches**, sans tenir compte du fait que la plupart sont partagées. La somme surcompte donc massivement. `docker system df` donne l'occupation réelle.

**Pourquoi.** Le stockage est adressé par contenu : une couche identique présente dans dix images n'existe qu'une fois sur le disque. Vingt images dérivées du même `eclipse-temurin:21-jre` affichent chacune ses 280 Mo de base, qui n'occupent pourtant que 280 Mo en tout.

**Nuance.** `docker system df` distingue `SIZE` (occupé) de `RECLAIMABLE` (récupérable si l'on supprime ce qui n'est utilisé par aucun conteneur). Avec `-v`, il détaille image par image et fait apparaître la colonne `SHARED SIZE`, la plus parlante des trois.

**Exemple.**

```
docker system df
# TYPE          TOTAL   ACTIVE   SIZE      RECLAIMABLE
# Images        15      12       8.667GB   330.5MB (3%)
docker system df -v | head -n 20 # détail, avec SHARED SIZE et UNIQUE SIZE
```

### Question 4 — Le secret « supprimé »

**Réponse.** Il a tort : `credentials.json` est **toujours dans l'image** et récupérable par quiconque la possède.

**Pourquoi.** Chaque instruction crée une couche immuable. La couche du `COPY` contient le fichier. La couche du `RUN` ne peut pas modifier la précédente : pour matérialiser la suppression, elle y écrit un marqueur d'effacement (*whiteout*, un fichier `.wh.credentials.json`). Le fichier est donc **masqué à l'exécution**, mais physiquement présent dans les données de l'image. Il suffit d'extraire l'archive pour le lire.

**Nuance.** Ne fonctionne pas non plus, contrairement à ce qu'on lit souvent : mettre `COPY` et `rm` sur la même ligne d'un `RUN` fonctionne (une seule couche), mais reste fragile car le secret transite dans le *build context*, apparaît dans les logs de build et dans le cache. Il ne faut pas non plus passer par `ARG` : `docker history` affiche les valeurs. Les vraies solutions sont le multi-stage (labo 05) et les *secret mounts* BuildKit (`RUN --mount=type=secret`), traités au labo 08.

**Exemple.**

```
docker save mon-image:1.0 -o /tmp/img.tar
mkdir /tmp/x && tar -xf /tmp/img.tar -C /tmp/x
grep -r "credentials" /tmp/x --include='*' -l # le fichier est là, dans une couche
```

### Question 5 — Push de 61 Mo pour une image de 310 Mo

**Réponse.** Le registry possède déjà toutes les couches communes aux deux versions (l'image de base, le JRE, les dépendances). Seule la couche qui a réellement changé — celle du JAR, 61 Mo — est transférée. Le registry recompose l'image à partir des couches qu'il détient déjà.

**Pourquoi.** Le `push` négocie couche par couche : le client demande au registry, pour chaque digest de couche, s'il le connaît (`HEAD /v2/<name>/blobs/<digest>`), et n'envoie que les manquantes. Même mécanisme au `pull`, d'où les `Already exists`.

**Nuance.** Si tout était retransféré à chaque fois, c'est que l'ordre des instructions du Dockerfile place les éléments variables **avant** les éléments stables : dès qu'une couche change, toutes celles qui la suivent sont invalidées et reconstruites, donc nouvelles. Le remède est l'ordre du plus stable au plus volatil (image de base → dépendances → code), et pour Spring Boot le découpage du JAR en couches ( `layertools` ) pour séparer les dépendances Maven, qui bougent rarement, du code applicatif, qui bouge à chaque commit. Sujet du labo 04.

### Exemple.

```
docker history mon-api:2.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
# une grosse couche de base + une petite couche COPY : c'est le bon profil
```

### Question 6 — Lecture de `docker images`

**Réponse.** Il y a **deux** images distinctes, pas trois. `api:2.0` et `api:1.9` ont le même `IMAGE ID` ( `f3a1b9c02d11` ) : c'est un seul objet portant deux étiquettes. La troisième ligne, `<none>`, est une image *dangling*.

**Pourquoi.** Un tag est une simple référence. `docker tag api:2.0 api:1.9` — ou un build qui réutilise l'identique — associe deux noms au même contenu. L'image `<none>` est l'ancienne image qui portait un de ces tags avant qu'il ne soit déplacé vers une nouvelle construction : elle a perdu son nom mais occupe toujours ses 295 Mo.

**Nuance.** `docker rmi api:1.9` ne supprimera **rien** sur le disque : Docker se contente de retirer l'étiquette (message `Untagged: api:1.9`), puisque `api:2.0` désigne encore l'image. On ne verra `Deleted: sha256:...` qu'à la disparition du dernier tag. C'est rassurant, mais aussi trompeur : croire avoir libéré de la place alors que non.

**Nuance bis — ce que vous verrez sur votre machine.** La sortie de l'énoncé est le format historique. Avec le magasin d'images **containerd** (celui de votre Docker Engine 29), la présentation par défaut change ( `IMAGE / ID / DISK USAGE / CONTENT SIZE` ) et les images *dangling* n'apparaissent quasiment plus : l'ancienne image est déréférencée puis nettoyée automatiquement. Le raisonnement sur les tags, lui, reste rigoureusement le même.

### Exemple.

```
docker tag nginx:alpine essai:v1
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep -E 'nginx|essai'
# nginx  alpine  4a73073bd557
# essai  v1      4a73073bd557      <- même IMAGE ID : une seule image
docker rmi essai:v1      # -> "Untagged: essai:v1" seulement
docker info | grep -i driver-type # quel magasin d'images ? containerd ou historique
```

### Question 7 — `exec format error`

**Réponse.** L'image a été construite pour `linux/arm64` (Apple Silicon) et le serveur de recette est en `linux/amd64`. Le noyau ne sait pas exécuter le binaire : d'où `exec format error`.

**Pourquoi.** Un tag peut référencer plusieurs architectures via une *manifest list*, mais un build local produit par défaut **une seule** architecture : celle de la machine qui construit. Pousser depuis un Mac M-series publie donc une image arm64 sous un tag que tout le monde croit universel.

**Nuance.** Dépannage immédiat : reconstruire en forçant la cible, `docker build --platform linux/amd64 ...`. Sur un Mac, cela passe par l'émulation QEMU : c'est **lent** (un build Maven peut tripler de durée) mais correct. Solution durable : ne plus jamais construire les images de production sur un poste de développement. La CI, qui tourne sur amd64, est la seule à pousser vers le registry ; les développeurs construisent localement pour eux seuls. Pour publier réellement les deux architectures, il faut `docker buildx build --platform linux/amd64,linux/arm64 --push`.

### Exemple.

```
docker image inspect --format '{{.Architecture}}/{{.Os}}' mon-api:1.0    # arm64/linux
docker run --rm --platform linux/amd64 alpine uname -m                  # x86_64
```

## Question 8 — `save` contre `export`

**Réponse.** `docker save` pour transporter une **image** ; `docker export` pour extraire le système de fichiers d'un **conteneur**. Pour un site isolé, c'est `save`.

**Pourquoi.** `save` archive les couches, le manifeste et la configuration : l'image rechargée est identique à l'originale, tags compris. `export` aplatit le système de fichiers d'un conteneur en une arborescence unique, sans historique de couches et **sans aucune métadonnée**.

**Nuance.** Avec `export`, on perd `ENTRYPOINT`, `CMD`, `ENV`, `WORKDIR`, `USER`, `EXPOSE`, `HEALTHCHECK`. Concrètement, une image Spring Boot ainsi transportée ne sait plus comment se lancer : `docker run` échouera faute de commande, et il faudra la réindiquer à la main à chaque exécution. On perd aussi le partage de couches, donc l'image réimportée est monolithique et le prochain transfert sera intégral.

### Exemple.

```
docker save -o nginx.tar nginx:alpine && ls -lh nginx.tar
docker rmi nginx:alpine && docker load -i nginx.tar    # tag et config restaurés

docker run -d --name tmp nginx:alpine
docker export tmp | docker import - nginx-aplati:v1
docker image inspect --format '{{json .Config.Cmd}}' nginx-aplati:v1    # null !
docker rm -f tmp && docker rmi nginx-aplati:v1
```

## Question 9 — Second `pull` instantané

**Réponse.** Docker a demandé au registry le **manifeste** du tag et comparé son digest à celui de l'image locale. Identiques : rien à télécharger. Seuls quelques kilo-octets de JSON ont circulé.

**Pourquoi.** Le manifeste est un petit document listant les digests des couches. Il suffit de le comparer pour savoir si quelque chose a changé, sans toucher aux centaines de Mo de données. Si un seul digest de couche différait, seule cette couche serait tirée.

**Nuance.** `Image is up to date` signifie « à jour **par rapport à ce tag, maintenant** », pas « c'est la dernière version du logiciel ». Et attention : `docker run` sans pull préalable n'interroge **pas** le registry si l'image est présente localement (politique par défaut `missing`). Une image locale obsolète peut donc servir des mois. En production, on force la politique : `docker run --pull=always`, ou `imagePullPolicy: Always` côté orchestrateur.

### Exemple.

```
docker pull nginx:alpine      # Status: Image is up to date for nginx:alpine
docker run --pull=always --rm nginx:alpine nginx -v    # vérifie le registry à chaque fois
```

### Question 10 — `toomanyrequests` en CI

**Réponse.** Docker Hub applique un quota de téléchargements aux utilisateurs anonymes, compté **par adresse IP**. Les agents de CI, tous derrière la même IP de sortie, épuisent le quota collectivement.

**Pourquoi.** Le quota est glissant sur plusieurs heures : le pipeline passe tant qu'il reste du crédit, échoue quand le seuil est atteint, repasse plus tard. D'où l'intermittence et l'impression que « ça vient de nulle part » — le pipeline n'a effectivement pas changé, c'est l'activité des voisins qui a changé.

**Nuance.** S'authentifier (`docker login`) relève le quota mais ne le supprime pas, et oblige à gérer un secret partagé. Les deux réponses d'entreprise sont : (1) un **pull-through cache** ou un miroir interne (Harbor, Nexus) qui ne tire d'Internet qu'une fois par image, (2) une **copie maîtrisée des images de base** dans le registry interne, avec un processus de mise à jour explicite. La seconde apporte en prime la maîtrise des versions et le scan de vulnérabilités.

#### Exemple.

```
# En CI, les images de base sont référencées ainsi :
FROM registry.masociete.fr/socle/eclipse-temurin:21-jre
# et jamais : FROM eclipse-temurin:21-jre
```

### Question 11 — Pourquoi pas `postgres:latest` en production

**Réponse.** Trois arguments de natures différentes :

1. **Reproductibilité.** `latest` est un pointeur mouvant. Deux déploiements de la même configuration peuvent installer deux versions majeures différentes de PostgreSQL. Le retour arrière devient impossible : rien n'identifie ce qui tournait avant.
2. **Disponibilité et souveraineté.** Dépendre de `docker.io` au moment du déploiement, c'est faire d'un service externe une dépendance critique : quota atteint, panne du Hub ou coupure du proxy, et la production ne redémarre plus. Le registry interne supprime cette dépendance.
3. **Sécurité et conformité.** Une image tirée directement d'Internet n'a été ni scannée, ni approuvée, ni tracée. L'entreprise doit pouvoir répondre à « quelle version, avec quelles CVE, tournait le 12 mars ? ». Un registry interne avec scan et rétention le permet ; `latest` non.

**Nuance.** Le fait que l'image soit *officielle* règle la question de la **qualité de la source**, pas celle du **contrôle de version** ni celle de la **chaîne d'approvisionnement**. Ce sont trois problèmes distincts, et `latest` n'en résout aucun.

#### Exemple.

```
# Ce qu'on écrit en production, tag figé côté registry interne :
image: registry.masociete.fr/socle/postgres:16.10-alpine
```

## Question 12 — `conflict: image is being used by stopped container`

**Réponse.** Un conteneur **arrêté mais non supprimé** référence encore cette image. Docker refuse de supprimer une image dont dépend un conteneur existant. La solution propre : supprimer d'abord le conteneur, puis l'image.

**Pourquoi.** Un conteneur arrêté conserve sa couche d'écriture, qui est empilée **par-dessus** les couches de l'image. Détruire l'image rendrait ce conteneur irrécupérable : ni `docker start`, ni `docker commit`, ni même l'inspection de ses fichiers ne fonctionneraient.

**Nuance.** `docker rmi -f` ne supprime pas vraiment les données dans ce cas : il se contente de retirer les **tags** et laisse l'image en `<none>` tant que le conteneur existe. On croit avoir nettoyé, on a en réalité fabriqué une image *dangling* et perdu l'information de ce qu'elle était — un conteneur qui affiche désormais un ID au lieu d'un nom d'image lisible. C'est un net recul pour le diagnostic.

### Exemple.

```
docker ps -a --filter ancestor=mon-api:1.0      # qui référence cette image ?
docker rm 4c2e9a1b7d33                          # d'abord le conteneur
docker rmi mon-api:1.0                          # ensuite l'image, sans -f
```

## Question 13 — Les couches à `0B`

**Réponse.** Ce sont les instructions qui ne touchent pas au système de fichiers mais seulement aux **métadonnées** de l'image : `ENV`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `LABEL`, `WORKDIR`, `USER`, `STOPSIGNAL`. Elles produisent une entrée d'historique sans données.

**Pourquoi.** Une image est faite de couches de fichiers **et** d'une configuration. Chaque instruction produit une entrée dans l'historique ; seules celles qui écrivent des fichiers produisent une couche non vide. Elles apparaissent quand même car l'historique retrace la construction complète.

**Nuance.** L'intérêt pratique est le suivant : `docker history` vous montre en une seconde **où passe le poids**. Une seule ligne à 180 Mo pour un `RUN apt-get install`, ou une couche `COPY .` énorme parce que le `.dockerignore` manque, se voient immédiatement. Attention toutefois : sur les images construites par BuildKit, la colonne `CREATED BY` est souvent tronquée ou vide pour les couches importées — croisez alors avec `docker image inspect` et le Dockerfile.

### Exemple.

```
docker history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}' | head -n 8
# 51.8MB  RUN /bin/sh -c set -x && apkArch=...
# 0B      ENV NJS_VERSION=1.0.0
# 0B      CMD ["nginx" "-g" "daemon off;"]
# 0B      EXPOSE map[80/tcp:{}]
```

## Question 14 — Trois stratégies de tags

**Réponse.**

| Stratégie                               | Retour arrière                                                                                              | Diagnostic d'incident                                                                                         |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| (a) <code>api:latest</code>             | <b>Impossible</b> : la version précédente n'a plus de nom, elle est peut-être déjà écrasée dans le registry | <b>Nul</b> : rien ne dit ce qui tourne ni depuis quand                                                        |
| (b) <code>api:1.4.2</code>              | <b>Possible</b> : il suffit de redéployer <code>1.4.1</code>                                                | <b>Correct</b> au niveau applicatif, mais un même numéro de version peut avoir été reconstruit plusieurs fois |
| (c) <code>api:1.4.2-b318-a9f3c21</code> | <b>Excellent</b> : chaque build est un artefact distinct et permanent                                       | <b>Excellent</b> : on remonte au commit exact et au job de CI en une lecture                                  |

**Pourquoi.** Un retour arrière consiste à redéployer un artefact **encore identifiable et présent**. Tout ce qui rend un tag réutilisable ou éphémère détruit cette capacité au pire moment.

**Nuance.** (c) est la bonne pratique, mais rarement seule : on publie en général des tags **mouvants** en plus des tags immuables — `api:1.4`, `api:stable` — pour que les environnements de développement suivent automatiquement. La règle est alors nette : les tags mouvants pour le confort, les tags immuables pour tout ce qui est déployé et audité. Attention enfin à la rétention du registry : un tag immuable ne sert à rien si la purge automatique supprime l'image au bout de trente jours.

### Exemple.

```
VERSION=1.4.2 ; BUILD=318 ; SHA=$(git rev-parse --short HEAD)
docker build -t registry.masociete.fr/paiement/api:$VERSION-b$BUILD-$SHA .
docker tag registry.masociete.fr/paiement/api:$VERSION-b$BUILD-$SHA \
registry.masociete.fr/paiement/api:$VERSION
```